COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 14 EXERCISES
MONGODB DBMS:
QUERYING ARRAYS

BOATRESERVATIONS MONGODB DATABASE

sailors collection

```
{sailorId: 10, sName: 'Emily', hkid: 'P936219(3)', age: 16},
{sailorId: 15, sName: 'Zoey', hkid: 'K091876(5)', rating: null, age: 17},
{sailorId: 22, sName: 'Dustin', hkid: 'A298456(6)', rating: 7, age: 45,
    reservations: [{boatId: 103, rDate: '08-SEP-25'},
                   {boatId: 104, rDate: '10-OCT-25'},<<<
                   {boatId: 101, rDate: '10-OCT-25'},
                   {boatId: 102, rDate: '10-NOV-25'}]},
{sailorId: 29, sName: 'Brutus', hkid: 'D746239(0)', rating: 1, age: 33},
{sailorId: 31, sName: 'Lucy', hkid: 'P930281(5)', rating: 10, age: 55,
    reservations: [{boatId: 102, rDate: '10-OCT-25'},
                   {boatId: 103, rDate: '06-NOV-25'},
                   {boatId: 102, rDate: '12-NOV-25'}]},
{sailorId: 32, sName: 'Andy', hkid: 'M028391(4)', rating: 8, age: 25},
{sailorId: 58, sName: 'Rachael', hkid: 'A819204(3)', rating: 10, age: 17},
{sailorId: 64, sName: 'Horatio', hkid: 'F710943(2)', rating: 7, age: 35,
    reservations: [{boatId: 101, rDate: '05-SEP-25'},
                   {boatId: 102, rDate: '08-SEP-25'}]},
{sailorId: 71, sName: 'Zorba', hkid: 'P910562(4)', rating: 1, age: 16},
{sailorId: 74, sName: 'Horatio', hkid: 'T810463(1)', rating: 9, age: 35,
    reservations: [{boatId: 103, rDate: '08-SEP-25'}]},
{sailorId: 85, sName: 'Amy', hkid: 'B719037(8)', rating: 3, age: 25},
{sailorId: 95, sName: 'Bob', hkid: 'E012735(5)', rating: 3, age: 63},
{sailorId: 99, sName: 'Chris', hkid: 'H814243(7)', rating: 10, age: 30,
    reservations: [{boatId: 102, rDate: '08-AUG-25'}]}
```

boats collection

```
{boatId: 101, bName: 'Interlake', color: 'blue', bType: 'dinghy',
    reservations: [{sailorId: 64, rDate: '05-SEP-25'},
                   {sailorId: 22, rDate: '10-OCT-25'}]},
{boatId: 102, bName: 'Interlake', color: 'red, bType: 'sloop',
    reservations: [{sailorId: 64, rDate: '08-SEP-25'},
                   {sailorId: 31, rDate: '10-OCT-25'},
                   {sailorId: 22, rDate: '10-NOV-25'},
                   {sailorId: 31, rDate: '12-NOV-25'},
                   {sailorId: 99, rDate: '08-AUG-25'}]},
{boatId: 103, bName: 'Clipper', color: 'green', bType: 'dinghy',
    reservations: [{sailorId: 22, rDate: '08-SEP-25'},
                   {sailorId: 74, rDate: '08-SEP-25'},
                   {sailorId: 31, rDate: '06-NOV-25'}]},
{boatId: 104, bName: 'Marine', color: 'red', bType: 'catamaran',
    reservations: [{sailorId: 22, rDate: '10-OCT-25'}]},<<<
{boatId: 105, bName: 'Serenity', color: 'cyan'}
```

EXERCISE 1

Find sailors who reserved boat 102 on September 8, 2025.
Include in the query result objects fields sailor id and name.

Query result: {sailorId: 64, sName: 'Horatio'}

Is this a correct solution?

No! Why?

```
db.sailors.find({'reservations.boatId': 102, 'reservations.rDate': '08-SEP-25'},
    {_id: 0, sailorId: 1, sName: 1});
```

Query result: {sailorId: 22, sName: 'Dustin'} {sailorId: 64, sName: 'Horatio'}

The implicit and condition can be satisfied in two different elements of a reservations array field.

☞ **Cannot use an and condition (either implicit or explicit) to match array elements!**

```
db.sailors.find({reservations: {$elemMatch: {boatId: 102, rDate: '08-SEP-25'}}},
    {_id: 0, sailorId: 1, sName: 1});
```

☞ **Need to use the $elemMatch operator when predicates need to be satisfied on the same array field element.**

EXERCISE 2

Find sailors who reserved boats 101 or 102 but not boats 103 or 104.
Include in the query result objects fields sailor id and name.

Query result: {sailorId: 64, sName: 'Horatio'} {sailorId: 99, sName: 'Chris'}

Is this a correct solution?

No! Why?

```
db.sailors.find({'reservations.boatId': {$in: [101, 102]},
                 'reservations.boatId': {$nin: [103, 104]}},
    {_id: 0, sailorId: 1, sName: 1});
```

Query result:

{sailorId: 10, sName: 'Emily'}
{sailorId: 29, sName: 'Brutus'}
{sailorId: 58, sName: 'Rachael'}
{sailorId: 71, sName: 'Zorba'}
{sailorId: 95, sName: 'Bob'}
{sailorId: 15, sName: 'Zoey'}
{sailorId: 32, sName: 'Andy'}
{sailorId: 64, sName: 'Horatio'}
{sailorId: 85, sName: 'Amy'}
{sailorId: 99, sName: 'Chris'}

The implicit and condition has two predicates on the same field reservations.boatId.

☞ **Only the last predicate is applied!**

EXERCISE 2 (CONTD)

Find sailors who reserved boats 101 or 102 but not boats 103 or 104.
Include in the query result objects fields sailor id and name.

Is this a correct solution?

No! Why?

```
db.sailors.find({reservations: {$elemMatch: {boatId: {$in: [101, 102]},
                                             boatId: {$nin: [103, 104]}}}},
    {_id: 0, sailorId: 1, sName: 1});
```

Query result: {sailorId: 22, sName: 'Dustin'} {sailorId: 31, sName: 'Lucy'} {sailorId: 64, sName: 'Horatio'} {sailorId: 99, sName: 'Chris'}

The query selects sailors who have reserved either boat 101 or 102!

If a boatId is equal to either 101 or 102 in an array element, then it cannot be equal to either 103 or 104!

☞ **The second condition is useless!**

EXERCISE 2 (CONTD)

Find sailors who reserved boats 101 or 102 but not boats 103 or 104.
Include in the query result objects fields sailor id and name.

```
db.sailors.find({$and: [{'reservations.boatId': {$in: [101, 102]}},
                        {'reservations.boatId': {$nin: [103, 104]}}]},
    {_id: 0, sailorId: 1, sName: 1});
```

☞ **The $and operator needs to be used.**

EXERCISE 3

Find reservations made for either boat 101 or boat 103.
Include in the query result objects fields sailor id, name and reservations.

Query result:

```
{sailorId: 22, sName: 'Dustin', reservations: [{boatId: 101, rDate: '10-OCT-25'},
                                               {boatId: 103, rDate: '08-SEP-25'}]}
{sailorId: 31, sName: 'Lucy', reservations: [{boatId: 103, rDate: '06-NOV-25'}]}
{sailorId: 64, sName: 'Horatio', reservations: [{boatId: 101, rDate: '05-SEP-25'}]}
{sailorId: 74, sName: 'Horatio', reservations: [{boatId: 103, rDate: '08SEP-25'}]}
```

Is this a correct solution?

Yes, but …

```
db.sailors.find({'reservations.boatId': {$in: [101, 103]}},
    {_id: 0, sailorId: 1, sName: 1, reservations: 1})
```

Query result:

```
{sailorId: 22, sName: 'Dustin', reservations: [{boatId: 103, rDate: '08-SEP-25'},
                                               {boatId: 104, rDate: '07-OCT-25'},
                                               {boatId: 102, rDate: '10-OCT-25'},
                                               {boatId: 101, rDate: '10-OCT-25'}]}
{sailorId: 31, sName: 'Lucy'}, reservations: [{boatId: 102, rDate: '10-NOV-25'},
                                              {boatId: 102, rDate: '12-NOV-25'},
                                              {boatId: 103, rDate: '06-NOV-25'}]}
{sailorId: 64, sName: 'Horatio', reservations: [{boatId: 101, rDate: '05-SEP-25'},
                                                {boatId: 102, rDate: '08-SEP-25'}]}
{sailorId: 74, sName: 'Horatio', reservations: [{boatId: 103, rDate: '08-SEP-25'}]}
```

How to show reservations only for boats 101 and 103 in the query result?

EXERCISE 3 (CONTD)

Find reservations made for either boat 101 or boat 103.
Include in the query result objects fields sailor id, name and reservations.

```
db.sailors.find({'reservations.boatId': {$in: [101, 103]}},
    {_id: 0, sailorId: 1, sName: 1,
        reservations: {$filter: {input: '$reservations',
                                 as: 'res',
                                 cond: {$in: ['$$res.boatId', [101, 103]]}}}});
```

Is the query filter condition {'reservations.boatId': {$in: [101, 103]} still required?

Yes! Why?

Otherwise, the query result shows all sailors with either a null or empty reservations field for those sailors who have not reserved boats 101 or 103.

EXERCISE 3 (CONTD)

Find reservations made for either boat 101 or boat 103.
Include in the query result objects fields sailor id, name and reservations.

```
db.sailors.find({'reservations.boatId': {$in: [101, 103]}},
    {_id: 0, sailorId: 1, sName: 1,
        reservations: {$filter: {input: '$reservations',
                                 as: 'res',
                                 cond: {$or: [{$eq: ['$$res.boatId', 101]},
                                              {$eq: ['$$res.boatId', 103]}]}}}});
```

If the **$or** operator is used in the **$filter** condition, then
the predicate needs to be specified as shown above.

EXERCISE 4

Find the number of reservations made by each sailor.
Include in the query result objects fields sailor id, name, rating and number of reservations made. Replace null ratings with 'none'.
Order the query result first by name ascending and then by sailor id ascending.

Query result:

```
{sailorId: 85, sName: 'Amy', rating: 3, numReservations: 0}
{sailorId: 32, sName: 'Andy', rating: 8, numReservations: 0}
{sailorId: 95, sName: 'Bob', rating: 3, numReservations: 0}
{sailorId: 29, sName: 'Brutus', rating: 1, numReservations: 0}
{sailorId: 99, sName: 'Chris', rating: 10, numReservations: 1}
{sailorId: 22, sName: 'Dustin', rating: 7, numReservations: 4}
{sailorId: 10, sName: 'Emily', rating: 'none', numReservations: 0}
{sailorId: 64, sName: 'Horatio', rating: 7, numReservations: 2}
{sailorId: 74, sName: 'Horatio', rating: 9, numReservations: 1}
{sailorId: 31, sName: 'Lucy', rating: 10, numReservations: 3}
{sailorId: 58, sName: 'Rachael', rating: 10, numReservations: 0}
{sailorId: 15, sName: 'Zoey', rating: 'none', numReservations: 0}
{sailorId: 71, sName: 'Zorba', rating: 1, numReservations: 0}
```

Is this a correct solution?

No! Why?

```
db.sailors.find({ },
    {_id: 0, sailorId: 1, sName: 1, rating: {$ifNull: ['$rating', 'none']},
        numReservations: {$size: '$reservations'}})
    .sort({sName: 1, sailorId: 1});
```

Query result:

Executor error during find command: boatReservationsdb.sailors :: caused by :: The argument to $size must be an array, but was of type: missing

EXERCISE 4 (CONT'D)

Find the number of reservations made by each sailor.
Include in the query result objects fields sailor id, name, rating and number of reservations made. Replace null ratings with 'none'.
Order the query result first by name ascending and then by sailor id ascending.

Is this a correct solution?

No! Why?

```
db.sailors.find({reservations: {$exists: true}},
    {_id: 0, sailorId: 1, sName: 1, rating: {$ifNull: ['$rating', 'none']},
        numReservations: {$size: '$reservations'}})
    .sort({sName: 1, sailorId: 1});
```

Query result:

```
{sailorId: 99, sName: 'Chris', rating: 10, numReservations: 1}
{sailorId: 22, sName: 'Dustin', rating: 7, numReservations: 4}
{sailorId: 64, sName: 'Horatio', rating: 7, numReservations: 2}
{sailorId: 74, sName: 'Horatio', rating: 9, numReservations: 1}
{sailorId: 31, sName: 'Lucy', rating: 10, numReservations: 3}
```

The query includes only sailors who have reserved a boat.

EXERCISE 4 (CONT'D)

Find the number of reservations made by each sailor.
Include in the query result objects fields sailor id, name, rating and number of reservations made. Replace null ratings with 'none'.
Order the query result first by name ascending and then by sailor id ascending.

```
db.sailors.find({ },
    {_id: 0, sailorId: 1, sName: 1, rating: {$ifNull: ['$rating', 'none']},
        numReservations: {$cond: {if: {$isArray: '$reservations'},
                                  then: {$size: '$reservations'},
                                  else: 0}}})
    .sort({sName: 1, sailorId: 1});
```

Notes

1. The rating field will be absent in the query result for sailors whose rating is null or absent if the $ifNull check is omitted.
2. It is not possible to sort on the numReservations field. Why?

EXERCISE 5

Find boats reserved by sailor 22 on October 10, 2025.
Include in the query result objects fields sailor id and name.

Query result: {boatId: 101, bName: 'Interlake'}

Is this a correct solution?

No! Why?

```
db.boats.find({'reservations.sailorId': 22, 'reservations.rDate': '10-OCT-25'},
    {_id: 0, boatId: 1, bName: 1});
```

Query result: {boatId: 101, bName: 'Interlake'} {boatId: 102, bName: 'Interlake'}

The implicit and condition can be satisfied in different elements of the reservations array field.

☞ **Cannot use and to match predicates on the same array field element.**

```
db.boats.find({reservations: {$elemMatch: {sailorId: 22, rDate: '10-OCT-25'}}},
    {_id: 0, boatId: 1, bName: 1});
```

☞ **Need to use the $elemMatch operator to match predicates on the same array field element.**

EXERCISE 6

Find sailors who made at least three reservations. Include in the query result objects fields sailor id, name and the third reservation.

Query result:
{sailorId: 22, sName: 'Dustin', 'third reservation': {boatId: 101, rDate: '10-OCT-25'}}
{sailorId: 31, sName: 'Lucy', 'third reservation': {boatId: 102, rDate: '12-NOV-25'}}

```
db.sailors.find({$expr: {$gte: [{$size: '$reservations'}, 3]}},
    {_id: 0, sailorId: 1, sName: 1, 'third reservation': {$arrayElemAt: ['$reservations', 2]}});
```

Is this a correct solution?

No! Why?

Query result:

Executor error during find command: boatReservationsdb.sailors :: caused by :: The argument to $size must be an array, but was of type: missing

If an array field can be absent, then you first need to check whether the array field exists before attempting to access it.

EXERCISE 6 (CONT'D)

Find sailors who made at least three reservations. Include in the query result objects fields sailor id, name and the third reservation.

Solution 1: Using array indexing and the **$arrayElemAt** operator

```
db.sailors.find({'reservations.2': {$exists: true}},
    {_id: 0, sailorId: 1, sName: 1, 'third reservation': {$arrayElemAt: ['$reservations', 2]}});
```

Solution 2: Using array indexing and the **$slice** operator

```
db.sailors.find({'reservations.2': {$exists: true}},
    {_id: 0, sailorId: 1, sName: 1, 'third reservation': {$slice: ['$reservations', 2, 1]}});
```

To rename an array field that you want to slice in a projection, the form shown above for the **$slice** operator needs to be used.

EXERCISE 7

Find any three sailors who are 16 years or older and have a rating that is among the lowest ratings for sailors 16 years or older. Include in the query result objects fields sailor name and rating. Order the result by rating ascending.

Query result:
{sName: 'Zorba', rating: 1}
{sName: 'Brutus', rating: 1}
{sName: 'Amy', rating: 3}

Is this a correct solution?

No! Why?

```
db.sailors.find({age: {$gte: 16}},
    {_id: 0, sName: 1, rating:1})
    .sort({rating: 1})
    .limit(3);
```

Query result:
{sName: 'Zoey', rating: null}
{sName: 'Emily'}
{sName: 'Brutus', rating: 1}

The query includes sailors who have a null or absent rating. Why?

☞ **Null and absent fields are the smallest values in MongoDB sort order.**

EXERCISE 7 (CONT'D)

Find any three sailors who are 16 years or older and have the lowest rating among sailors 16 years or older.
Include in the query result objects fields sailor name and rating.
Order the result by rating ascending.

How about this solution?

```
db.sailors.find({age: {$gte: 16}, rating: {$exists: true}},
    {_id: 0, sName: 1, rating:1})
    .sort({rating: 1})
    .limit(3);
```

Query result:
```
{sName: 'Zoey', rating: null}
{sName: 'Zorba', rating: 1}
{sName: 'Brutus', rating: 1}
```

The query still includes sailors who have a null rating.

☞ **Null and absent fields are the smallest values in MongoDB sort order.**

EXERCISE 7 (CONT'D)

Find any three sailors who are 16 years or older and have the lowest rating among sailors 16 years or older.
Include in the query result objects fields sailor name and rating.
Order the result by rating ascending.

```
db.sailors.find({age: {$gte: 16}, rating: {$ne: null}},
    {_id: 0, sName: 1, rating:1})
    .sort({rating: 1})
    .limit(3);
```